Tabular RDF Mapping

Simon Frost

Overview

RDFLib.jl includes a tabular mapping module — inspired by maplib — that converts Tables.jl-compatible data to RDF and queries it with SPARQL, returning tabular results. Any Tables.jl source works: `Vector{NamedTuple}`, DataFrames, CSV files, etc.

```
using RDFLib
```

Quick Start with map_default!

The fastest way to get tabular data into RDF. Each column becomes a predicate; one column is the subject.

```
people = [
    (id = "alice", name = "Alice", age = 30),
    (id = "bob",   name = "Bob",   age = 25),
    (id = "carol", name = "Carol", age = 35),
]

m = RDFMapping()
tpl = map_default!(m, people, :id;
    subject_prefix   = "http://example.org/person/",
    predicate_prefix = "http://example.org/",
    types = [URIRef("http://example.org/Person")])

println("Mapped $(length(m)) triples from $(length(people)) rows")
```

```
Mapped 9 triples from 3 rows
```

Each row produces a `rdf:type` triple plus one per data column:

```
println(serialize(m, TurtleFormat()))
```

```
@prefix ns1: <http://example.org/person/> .
@prefix ns2: <http://example.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ns1:alice a ns2:Person ;
    ns2:age 30 ;
    ns2:name "Alice" .

ns1:bob a ns2:Person ;
    ns2:age 25 ;
    ns2:name "Bob" .

ns1:carol a ns2:Person ;
    ns2:age 35 ;
    ns2:name "Carol" .
```

SPARQL Querying

Query the mapped graph and get tabular results back:

```
results = rdf_query(m, """
    PREFIX ex: <http://example.org/>
    SELECT ?person ?name ?age WHERE {
        ?person a ex:Person .
        ?person ex:name ?name .
        ?person ex:age ?age .
    }
    ORDER BY ?name
""")
for row in sparql_query(m.graph, """
    PREFIX ex: <http://example.org/>
    SELECT ?name ?age WHERE {
        ?person a ex:Person .
```

```
        ?person ex:name ?name .
        ?person ex:age ?age .
    }
    ORDER BY ?name
""")
    println("  $(row["name"]) - age $(row["age"])")
end
```

```
  Literal("Alice") - age Literal("30", datatype=URIRef("http://www.w3.org/2001/XMLSchema#inte
  Literal("Bob") - age Literal("25", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integ
  Literal("Carol") - age Literal("35", datatype=URIRef("http://www.w3.org/2001/XMLSchema#inte
```

Custom Templates with RDFTemplate

For full control over the mapping, define a `RDFTemplate`:

```
scientists = [
    (id = 1, name = "Marie Curie",    field = "Physics",   born = 1867),
    (id = 2, name = "Ada Lovelace",   field = "Computing", born = 1815),
    (id = 3, name = "Rosalind Franklin", field = "Chemistry", born = 1920),
]

tpl = RDFTemplate(
    subject = :id,
    subject_prefix = "http://example.org/scientist/",
    properties = [
        (URIRef("http://xmlns.com/foaf/0.1/name"), :name, AutoColumn()),
        (URIRef("http://example.org/field"), :field, AutoColumn()),
        (URIRef("http://example.org/birthYear"), :born, LiteralColumn(XSD.integer)),
    ],
    types = [URIRef("http://example.org/Scientist")]
)

m2 = RDFMapping()
rdf_map!(m2, scientists, tpl)
println("Mapped $(length(m2)) triples")
println(serialize(m2, TurtleFormat()))
```

```
Mapped 12 triples
@prefix ns1: <http://example.org/scientist/> .
```

```
@prefix ns2: <http://example.org/> .
@prefix ns3: <http://xmlns.com/foaf/0.1/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ns1:1 a ns2:Scientist ;
    ns2:birthYear 1867 ;
    ns2:field "Physics" ;
    ns3:name "Marie Curie" .

ns1:2 a ns2:Scientist ;
    ns2:birthYear 1815 ;
    ns2:field "Computing" ;
    ns3:name "Ada Lovelace" .

ns1:3 a ns2:Scientist ;
    ns2:birthYear 1920 ;
    ns2:field "Chemistry" ;
    ns3:name "Rosalind Franklin" .
```

Column Type Hints

Control how column values map to RDF terms:

```
m3 = RDFMapping()
data = [
    (id = "alice", homepage = "http://alice.example.org", bio = "A researcher", score = 4.5)
]

tpl = RDFTemplate(
    subject = :id,
    subject_prefix = "http://example.org/person/",
    properties = [
        # IRIColumn: values become URIRef nodes
        (URIRef("http://xmlns.com/foaf/0.1/homepage"), :homepage, IRIColumn()),
        # LiteralColumn: explicit XSD datatype
        (URIRef("http://example.org/score"), :score, LiteralColumn(XSD.double)),
        # LangColumn: language-tagged string
        (URIRef("http://example.org/bio"), :bio, LangColumn("en")),
```

```
        # AutoColumn (default): auto-detect from Julia type
    ]
)
rdf_map!(m3, data, tpl)
println(serialize(m3, TurtleFormat()))
```

```
@prefix ns1: <http://example.org/person/> .
@prefix ns2: <http://xmlns.com/foaf/0.1/> .
@prefix ns3: <http://> .
@prefix ns4: <http://example.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ns1:alice ns4:bio "A researcher"@en ;
    ns4:score 4.5e0 ;
    ns2:homepage ns3:alice.example.org .
```

Column Type	Usage	Example Output
`AutoColumn()`	Detect from Julia type	`"42"^^xsd:integer`
`IRIColumn()`	Value becomes a URI	`<http://...>`
`LiteralColumn(XSD.integer)`	Explicit datatype	`"42"^^xsd:integer`
`LangColumn("en")`	Language tag	`"hello"@en`

OTTR Templates

OTTR (Reasonable Ontology Templates) provides a powerful template language for reusable RDF mappings. RDFLib.jl supports the stOTTR serialization format.

Basic OTTR Template

```
m4 = RDFMapping()

add_template!(m4, """
    @prefix ex: <http://example.org/> .
    @prefix foaf: <http://xmlns.com/foaf/0.1/> .

    ex:PersonTemplate [?id, ?name, ?age] :: {
        ottr:Triple(ex:person/{?id}, a, foaf:Person),
        ottr:Triple(ex:person/{?id}, foaf:name, ?name),
        ottr:Triple(ex:person/{?id}, foaf:age, ?age)
    } .
""")

data = [
    (id = 1, name = "Alice", age = 30),
    (id = 2, name = "Bob",   age = 25),
    (id = 3, name = "Carol", age = 35),
]

ottr_map!(m4, "http://example.org/PersonTemplate", data)
println("OTTR mapped $(length(m4)) triples")
println(serialize(m4, TurtleFormat()))
```

```
OTTR mapped 9 triples
@prefix ns1: <http://example.org/> .
@prefix ns2: <http://xmlns.com/foaf/0.1/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ns1:person <1> rdf:type,
        ns2:name,
        ns2:age ;
    <2> rdf:type,
        ns2:name,
        ns2:age ;
    <3> rdf:type,
        ns2:name,
        ns2:age .
```

OTTR with Constants and Type Shorthand

Templates can include constant values and use `a` for `rdf:type`:

```
m5 = RDFMapping()

add_template!(m5, """
    @prefix ex: <http://example.org/> .
    @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

    ex:SensorReading [?sensor_id, ?value, ?unit] :: {
        ottr:Triple(ex:reading/{?sensor_id}, a, ex:Measurement),
        ottr:Triple(ex:reading/{?sensor_id}, ex:value, ?value),
        ottr:Triple(ex:reading/{?sensor_id}, ex:unit, ?unit),
        ottr:Triple(ex:reading/{?sensor_id}, ex:source, "automatic"^^xsd:string)
    } .
""")

readings = [
    (sensor_id = "s1", value = 22.5, unit = "celsius"),
    (sensor_id = "s2", value = 1013.25, unit = "hPa"),
]

ottr_map!(m5, "http://example.org/SensorReading", readings)
println("Sensor data: $(length(m5)) triples")
```

```
Sensor data: 8 triples
```

SPARQL INSERT

Derive new triples from existing ones using SPARQL CONSTRUCT:

```
m6 = RDFMapping()
map_default!(m6, people, :id;
    subject_prefix   = "http://example.org/person/",
    predicate_prefix = "http://example.org/",
    types = [URIRef("http://example.org/Person")])

rdf_insert!(m6, """
    PREFIX ex: <http://example.org/>
    CONSTRUCT {
```

```
        ?person ex:label ?name
    } WHERE {
        ?person ex:name ?name .
    }
""")

println("After INSERT: $(length(m6)) triples")
```

```
After INSERT: 12 triples
```

SHACL Validation Integration

Validate mapped data against SHACL shapes:

```
m_val = RDFMapping()
map_default!(m_val, people, :id;
    subject_prefix   = "http://example.org/person/",
    predicate_prefix = "http://example.org/",
    types = [URIRef("http://example.org/Person")])

shapes_ttl = """
    @prefix sh: <http://www.w3.org/ns/shacl#> .
    @prefix ex: <http://example.org/> .
    @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

    ex:PersonShape a sh:NodeShape ;
        sh:targetClass ex:Person ;
        sh:property [
            sh:path ex:name ;
            sh:minCount 1 ;
        ] ;
        sh:property [
            sh:path ex:age ;
            sh:minCount 1 ;
        ] .
"""

report = rdf_validate(m_val, shapes_ttl)
println("Validation conforms: $(report.conforms)")
```

```
Validation conforms: true
```

Datalog Reasoning Integration

Apply Datalog-style inference to derive new facts:

```
m_dl = RDFMapping()

# Map organizational data
org_data = [
    (id = "alice", manages = "bob"),
    (id = "bob",   manages = "carol"),
]

for row in org_data
    subj = URIRef("http://example.org/person/$(row.id)")
    obj  = URIRef("http://example.org/person/$(row.manages)")
    add!(m_dl.graph, Triple(subj, URIRef("http://example.org/manages"), obj))
end

println("Before reasoning: $(length(m_dl)) triples")
rdf_reason!(m_dl)
println("After reasoning: $(length(m_dl)) triples")
```

```
Before reasoning: 2 triples
After reasoning: 2 triples
```

Performance

RDFLib.jl's mapping engine is optimized for bulk operations:

- **Deferred indexing**: index building is deferred until a query is issued
- **Skip dedup**: when the store is empty, duplicate checking is skipped
- **Pre-allocated vectors**: `sizehint!` reduces memory reallocation
- **Integer subject fast-path**: numeric IDs avoid string encoding overhead

These optimizations make RDFLib.jl **1.6–3.5× faster** than maplib (Rust backend) on mapping benchmarks across 1K–100K rows.

API Summary

Function	Purpose
`RDFMapping()`	Create mapping context
`map_default!(m, table, :col)`	Auto-map columns to predicates
`rdf_map!(m, table, template)`	Map using explicit template
`rdf_query(m, sparql)`	SPARQL SELECT → NamedTuple
`rdf_insert!(m, sparql)`	SPARQL CONSTRUCT → insert triples
`rdf_update!(m, sparql)`	SPARQL UPDATE
`add_template!(m, stottr)`	Register OTTR template
`ottr_map!(m, iri, table)`	Expand OTTR template
`rdf_validate(m, shapes)`	SHACL validation
`rdf_reason!(m)`	Datalog reasoning
`serialize(m, format)`	Serialize graph